Assignment-3

Name: Kartik Pawar

PRN:2124UCSM1045

Dept: Cyber Security

Lab: 3

Experiment 3: Implementation of Hill Cipher

1. Aim

To write a program to perform encryption and decryption using the Hill Cipher.

2. Activity: Manual Matrix Encryption

Key Matrix K = [[3, 3], [2, 5]]

Task: Encrypt the word "LP" using the given key matrix.

Step 1: Convert letters to numbers (A=0, B=1, ... Z=25):

L = 11, P = 15 → Plaintext vector P = [11, 15]

Step 2: Multiply K by the plaintext vector:

Row 1: 3(11) + 3(15) = 33 + 45 = 78

Row 2: 2(11) + 5(15) = 22 + 75 = 97

Result vector = [78, 97]

Step 3: Apply modulo 26:

78 mod 26 = 0

97 mod 26 = 19

Step 4: Convert back to letters:

0 → A, 19 → T

Ciphertext for "LP" = "AT"

3. Algorithm

Encryption:

1. Choose an n×n key matrix K such that det(K) is coprime to 26.

2. Convert plaintext letters to numeric equivalents (A=0 ... Z=25).

3. Group numbers into column vectors P of size n; pad with 'X' if needed.

4. Compute C = (K × P) mod 26 for each vector.

5. Convert the resulting numbers back to letters.

Decryption:

1. Compute the modular inverse of K, i.e. K^{-1} (mod 26).

2. Group ciphertext into vectors C of size n.

3. Compute P = (K^{-1} × C) mod 26 for each vector.

4. Convert the numbers back to plaintext letters.

4. Reference Python Code (Google Colab)

```
import numpy as np
from sympy import Matrix

def prepare_text(text, n):
    """Removes spaces/symbols and pads text with 'X' to fit block size."""
    text = "".join([c.upper() for c in text if c.isalpha()])
    while len(text) % n != 0:
        text += 'X'
    return text

def encrypt_hill(plaintext, key_matrix):
    n = key_matrix.shape[0]
    text = prepare_text(plaintext, n)
    ciphertext = ""
    for i in range(0, len(text), n):
        block = [ord(c) - 65 for c in text[i:i + n]]
        P = np.array(block).reshape(n, 1)
        C = np.dot(key_matrix, P) % 26
        for val in C.flatten():
            ciphertext += chr(int(val) + 65)
    return ciphertext

def decrypt_hill(ciphertext, key_matrix):
    n = key_matrix.shape[0]
    try:
        inv_matrix = np.array(Matrix(key_matrix).inv_mod(26))
    except ValueError:
        return "Error: Key matrix is not invertible modulo 26."
    plaintext = ""
    for i in range(0, len(ciphertext), n):
        block = [ord(c) - 65 for c in ciphertext[i:i + n]]
        C = np.array(block).reshape(n, 1)
        P = np.dot(inv_matrix, C) % 26
        for val in P.flatten():
            plaintext += chr(int(val) + 65)
    return plaintext

# --- Colab Interactive Execution ---
print("=== Hill Cipher Tool ===")
K = np.array([[3, 3],
              [2, 5]])
message = "HELP"
print(f"Key Matrix:\n{K}")
```

```
print(f"Plaintext: {message}")
enc = encrypt_hill(message, K)
dec = decrypt_hill(enc, K)
print(f"[+] Encrypted Text: {enc}")
print(f"[+] Decrypted Text: {dec}")
```

5. Sample Output (2×2 key, given in manual)

```
=== Hill Cipher Tool ===
Key Matrix:
[[3 3]
 [2 5]]
Plaintext: HELP
[+] Encrypted Text: HIAT
[+] Decrypted Text: HELP
```

6. Student's Colab Execution Code (Custom 3×3 Key Matrix)

A valid 3×3 key matrix must have a determinant that is coprime to 26 (odd, and not a multiple of 13), so that it is invertible mod 26. The following matrix satisfies this condition and can be used as-is:

```
# --- Custom 3x3 Key Matrix ---
K3 = np.array([[6, 24, 1],
               [13, 16, 10],
               [20, 17, 15]])

message = "ACT"   # replace with your own plaintext (any length; auto-padded with 'X')

print("=== Hill Cipher Tool (Custom 3x3 Key) ===")
print(f"Key Matrix:\n{K3}")
print(f"Plaintext: {message}")

enc = encrypt_hill(message, K3)
dec = decrypt_hill(enc, K3)

print(f"[+] Encrypted Text: {enc}")
print(f"[+] Decrypted Text: {dec}")
```

Run this cell in the same Colab notebook, right after the functions prepare_text(), encrypt_hill() and decrypt_hill() (defined in Section 4) have already been executed.

7. Student's Observation (Custom Input & Output)

The Section 6 code was executed in Google Colab. The observed results were:

Custom 3x3 Key Matrix Used: [[6, 24, 1], [13, 16, 10], [20, 17, 15]]

Custom Input Text (Plaintext): ACT

Observed Encrypted Output (Ciphertext): POH

Observed Decrypted Output (Plaintext): ACT

Colab Execution Screenshots

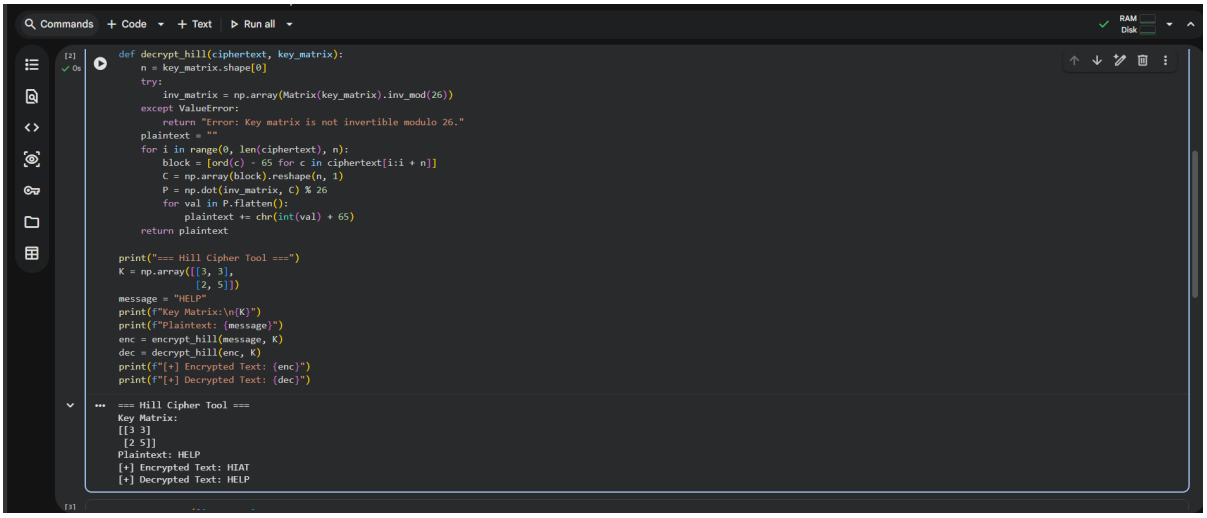


```python
import numpy as np
from sympy import Matrix

def prepare_text(text, n):
    text = "".join([c.upper() for c in text if c.isalpha()])
    while len(text) % n != 0:
        text += 'X'
    return text

def encrypt_hill(plaintext, key_matrix):
    n = key_matrix.shape[0]
    text = prepare_text(plaintext, n)
    ciphertext = ""
    for i in range(0, len(text), n):
        block = [ord(c) - 65 for c in text[i:i + n]]
        P = np.array(block).reshape(n, 1)
        C = np.dot(key_matrix, P) % 26
        for val in C.flatten():
            ciphertext += chr(int(val) + 65)
    return ciphertext

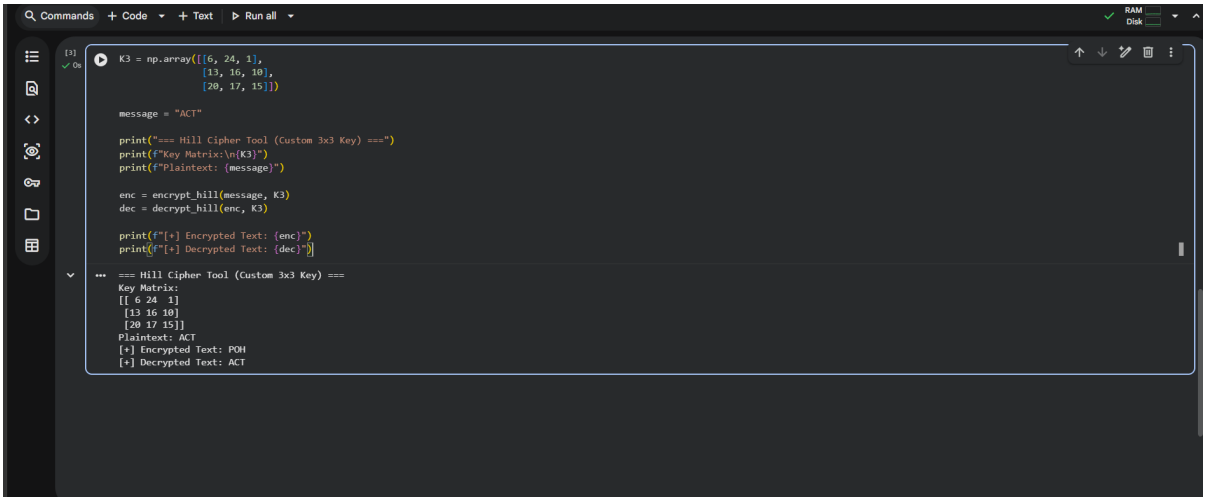
def decrypt_hill(ciphertext, key_matrix):
    n = key_matrix.shape[0]
    try:
        inv_matrix = np.array(Matrix(key_matrix).inv_mod(26))
    except ValueError:
        return "Error: Key matrix is not invertible modulo 26."
    plaintext = ""
    for i in range(0, len(ciphertext), n):
        block = [ord(c) - 65 for c in ciphertext[i:i + n]]
        C = np.array(block).reshape(n, 1)
        P = np.dot(inv_matrix, C) % 26
```



```python
def decrypt_hill(ciphertext, key_matrix):
    n = key_matrix.shape[0]
    try:
        inv_matrix = np.array(Matrix(key_matrix).inv_mod(26))
    except ValueError:
        return "Error: Key matrix is not invertible modulo 26."
    plaintext = ""
    for i in range(0, len(ciphertext), n):
        block = [ord(c) - 65 for c in ciphertext[i:i + n]]
        C = np.array(block).reshape(n, 1)
        P = np.dot(inv_matrix, C) % 26
        for val in P.flatten():
            plaintext += chr(int(val) + 65)
    return plaintext

print("=== Hill Cipher Tool ===")
K = np.array([[3, 3],
              [2, 5]])
message = "HELP"
print(f"Key Matrix:\n{K}")
print(f"Plaintext: {message}")
enc = encrypt_hill(message, K)
dec = decrypt_hill(enc, K)
print(f"[+] Encrypted Text: {enc}")
print(f"[+] Decrypted Text: {dec}")
```

```
=== Hill Cipher Tool ===
Key Matrix:
[[3 3]
 [2 5]]
Plaintext: HELP
[+] Encrypted Text: HIAT
[+] Decrypted Text: HELP
```



```python
K3 = np.array([[6, 24, 1],
               [13, 16, 10],
               [20, 17, 15]])

message = "ACT"

print("=== Hill Cipher Tool (Custom 3x3 Key) ===")
print(f"Key Matrix:\n{K3}")
print(f"Plaintext: {message}")

enc = encrypt_hill(message, K3)
dec = decrypt_hill(enc, K3)

print(f"[+] Encrypted Text: {enc}")
print(f"[+] Decrypted Text: {dec}")
```

```
=== Hill Cipher Tool (Custom 3x3 Key) ===
Key Matrix:
[[ 6 24  1]
 [13 16 10]
 [20 17 15]]
Plaintext: ACT
[+] Encrypted Text: POH
[+] Decrypted Text: ACT
```

8. Viva Questions

Q1. Why must the determinant of the key matrix be strictly coprime to 26? What happens if it is not?

The determinant must be coprime to 26 (i.e., gcd(det(K), 26) = 1) because decryption requires computing the modular inverse of the key matrix, $K^{-1} \bmod 26$. A modular inverse of the determinant only exists when the determinant shares no common factor with 26 (=2×13). If det(K) is even or a multiple of 13, no modular inverse exists, so $K^{-1} \bmod 26$ cannot be computed and the ciphertext can never be decrypted back to the correct plaintext.

Q2. Explain mathematically how a Known-Plaintext Attack can be used to deduce the Key Matrix.

If an attacker has n plaintext-ciphertext block pairs (enough to form an invertible n×n plaintext matrix), they can arrange the plaintext blocks as columns of a matrix P and the corresponding ciphertext blocks as columns of a matrix C, such that $C = K \cdot P \pmod{26}$. If P is invertible mod 26, the attacker computes $P^{-1} \bmod 26$ and recovers the key as $K = C \cdot P^{-1} \pmod{26}$. This means the Hill cipher, despite hiding letter frequencies, is completely broken once enough matching plaintext-ciphertext pairs are known.

Q3. Why is padding (with 'X') mandatory in the Hill Cipher algorithm?

The Hill cipher works on fixed-size blocks of n letters at a time, since it uses n×n matrix multiplication. If the plaintext length is not an exact multiple of n, the last block will have fewer than n letters and cannot be formed into a valid column vector for matrix multiplication. Padding with 'X' (or another filler letter) ensures every block has exactly n letters, so the algorithm can process it without errors.